

ProjetoAngularFirestore

Audit date - 2026-05-08 18:25

Angular ^16.0.0 - 18 TS files - 676 LOC

50/100

Grade D

Faible - problemes significatifs, refactoring urgent conseil.

23 issues detected

2

CRITICAL

18

IMPORTANT

3

MINOR

Project overview

Angular version	^16.0.0
TypeScript files	18
HTML templates	3
Components / Services / Modules	2 / 3 / 4
Pipes / Guards	0 / 0
Total lines of code	676
Tests detected	Yes

Severity summary



Critical: 2

Important: 18

Minor: 3

Issues detail

Securite

CRITICAL

SEC002 Cle API ou secret hardcoded (2)

Une cle API, token ou secret hardcoded dans le code source est expose des qu'il est commit. Toute personne ayant acces au repo (ou au bundle prod) peut l'extraire et abuser des credentials. Cas reel : OpenAI revoque automatiquement les sk-... detectees sur GitHub public.

Fix: Stocker dans `src/environments/environment.ts` (ignore par .gitignore) ou via variables d'env injectees au build (`ng build --configuration=production` + `fileReplacements`). Pour les secrets serveur, ne jamais les inclure cote client : passer par un backend proxy. Si la cle a deja ete commit, il faut la revoquer immediatement (rotate) puis purger l'historique git.

OCCURRENCES

[src/environments/environment.prod.ts:3](#)

```
apiKey: "AIzaSyCGdwCWx_8o2hYEmLBhK6xQzCmp5h12gzk",
```

[src/environments/environment.ts:7](#)

```
apiKey: "AIzaSyCGdwCWx_8o2hYEmLBhK6xQzCmp5h12gzk",
```

Type Safety

IMPORTANT

TYPE001 Usage de 'any' TypeScript (16)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

Fix: Typer explicitement (interface, type, générique). Utiliser `unknown` si le type est vraiment inconnu.

OCCURRENCES

[src/app/home/home.page.ts:15](#)

```
pokemon:any = {
```

[src/app/services/crud.service.ts:29](#)

```
insert(item: any, remoteCollectionName: string): Boolean {
```

[src/app/services/crud.service.ts:61](#)

```
let data: any = [];
```

[src/app/services/crud.service.ts:73](#)

```
.catch((_ : any) => {
```

[src/app/services/crud.service.ts:88](#)

```
async fetchByOperatorParam(fieldName: string, operator: WhereFilterOp, fieldValue: any, remoteC...
```

[src/app/services/crud.service.ts:91](#)

```
let data: any = [];
```

[src/app/services/crud.service.ts:103](#)

```
.catch((_: any) => {
```

[src/app/services/crud.service.ts:121](#)

```
let data: any = [];
```

... and 8 more occurrence(s).

IMPORTANT

TYPE002 Cast 'as any' explicite (1)

Un cast `as any` desactive volontairement TypeScript pour cette expression. Different de `: any` (TYPE001) qui declare un type ambigu : `as any` est un acte explicite de bypass du verificateur. Apparaît souvent quand un dev se bat avec un type de librairie tiers, ou quand un payload API renvoie une structure non typee. Le probleme : le compilateur ne peut plus garantir que les acces suivants (`.foo`, `.bar()`) sont valides - un refactor de la source ne mettra plus a jour les usages, et un null/undefined dans le payload ne sera pas detecte au build.

Fix: 1) Si la forme est connue : declarer une `interface` ou `type` et caster vers ce type (`as User`). 2) Si la forme est partiellement connue : `as Partial<User>` ou `as Pick<User, 'id'|'name'>`. 3) Si la forme est vraiment inconnue : caster vers `unknown` puis valider avec un type guard avant l'acces (`if (typeof x === 'object' && x !== null && 'id' in x)`). 4) Pour les reponses HTTP : utiliser `http.get<MyType>(url)` ou un schema runtime (zod, yup) pour valider la forme avant cast. Le cast `as unknown` est preferable a `as any` car il force au moins une etape de validation explicite.

OCCURRENCES

[src/zone-flags.ts:6](#)

```
(window as any).__Zone_disable_customElements = true;
```

Performance

IMPORTANT

PERF002 Route sans lazy loading (1)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

Fix: Remplacer `component: MyComponent` par `loadComponent: () => import('./my.component').then(m => m.MyComponent)`

OCCURRENCES

[src/app/home/home-routing.module.ts:8](#)

```
component: HomePage,
```

Code Quality

MINOR

DEBUG001 console.log en production (3)

Les console.log oubliés exposent des données internes en prod et polluent la console.

Fix: Supprimer ou remplacer par un service de logging. Configurer `build.optimization.scripts` pour stripper en prod.

OCCURRENCES

[src/main.ts:12](#)

```
.catch(err => console.log(err));
```

[src/app/services/crud.service.ts:30](#)

```
console.log(item)
```

[src/app/services/auth.service.ts:55](#)

```
console.log(response.user);
```

Lazy loading

Eager routes (no lazy)	1
Lazy routes	1
Lazy loading ratio	50%

Refactoring plan

This week - Critical

- Cle API ou secret hardcoded (SEC002)

Une cle API, token ou secret hardcoded dans le code source est expose des qu'il est commit. Toute personne ayant acces au repo (...)

This month - Important

- Usage de 'any' TypeScript (TYPE001)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

- Cast 'as any' explicite (TYPE002)

Un cast `as any` desactive volontairement TypeScript pour cette expression. Different de `: any` (TYPE001) qui declare un type ...

- Route sans lazy loading (PERF002)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

On the roadmap - Minor

- console.log en production (DEBUG001)

Les console.log oubliés exposent des données internes en prod et polluent la console.

This report is produced by automated static analysis. It does not replace a thorough manual review by an experienced Angular developer.